

Embedded and Edge AI Project Report

Riccardo Masetti
riccardo1.masetti@mail.polimi.it

Samuele Tondelli
samuele.tondelli@mail.polimi.it

I. INTRODUCTION

Indoor human localization is an important task for several applications and settings, including smart environments, human activity monitoring, and context-aware embedded systems. In this project, carried out as part of the Embedded and Edge Artificial Intelligence course at Politecnico di Milano, we address the problem of estimating the two-dimensional positions of multiple people inside a room, using Ultra-Wideband (UWB) radar signals, which offer advantages over vision-based systems and GPS by avoiding visual data acquisition and supporting accurate localization in indoor environments [1].

The main objective of the project is to design a compact TinyML pipeline that processes raw UWB signals and predicts the 2D positions of up to four people in each frame, while satisfying the memory, size, and operation constraints for a microcontroller-based deployment [2].

This report describes the main steps of the project, from problem and data analysis to preprocessing, model design, experiments and final evaluation, explaining and justifying the main design choices made throughout the development process.

II. PROBLEM SETTING

This section introduces the localization problem addressed in the project, focusing on how the data were collected, how the dataset is structured, and which main constraints had to be considered during the development of the solution.

A. Data Collection Settings

The setting is composed by an indoor room of size 4.8 m x 7.2 m. Data collection is performed thanks to six fixed UWB radar sensors placed around the room perimeter at a height of 1.2 m. Each radar contains three antennas and records Channel Impulse Response (CIR) signals over 120 range bins, sampled at 25 Hz. The use of multiple radars provides different views of the same environment, while the antennas add angular diversity to the collected signals.

The training dataset is organized in 24 windows, each lasting 300 seconds (7500 frames per window) for a total of 120 minutes of data (180000 frames). The recorded scenarios include different numbers of people, from an empty room up to four subjects, and different behaviours such as random walking, seated people and serpentine walks. However, the number of windows available for each number of people is not balanced, as shown in Table I. This required particular care when splitting the data into training, validation, and test sets, in order to preserve a reasonable representation of all scenarios.

TABLE I

DISTRIBUTION OF ACQUISITION WINDOWS BY NUMBER OF PEOPLE IN THE ROOM.

People	Window indices	Windows
0	22, 23	2
1	16, 17, 18, 19, 20, 21	6
2	0, 1, 2, 3, 4	5
3	12, 13, 14, 15	4
4	5, 6, 7, 8, 9, 10, 11	7

B. Data Format

Each acquisition window is stored as a .npz file containing four arrays: the raw radar measurements, the ground-truth positions, a validity mask, and the timestamps. The role and shape of each array are the following:

- `radar_cir_iq`: with shape $(T, 6, 3, 120, 2)$, contains the raw complex CIR measurements. The dimensions represent, respectively, the number of frames, the six radars, the three antennas per radar, the 120 range bins, and the In-phase and Quadrature components.
- `people_xy`: with shape $(T, 4, 2)$, contains the ground-truth 2D positions of up to four people for each frame, expressed as (x, y) coordinates in meters.
- `people_mask`: with shape $(T, 4)$, indicates which of the four possible position entries are valid. This is needed because each frame can contain fewer than four people.
- `timestamps`: with shape (T) , stores the acquisition timestamp associated with each frame.

C. Main Challenges

The main difficulty of the problem comes from the variable number of people in the room, including the empty-room case. That is because the model doesn't have only to learn to localize people, but also to predict how many people are present on the scene [3]. The model must avoid false positives when no person is present, while also avoiding missed detections when multiple people are in the scene, and that can be difficult in particular when people are close to each other.

D. Constraints

The project specification also imposed strict constraints related to the target embedded deployment. The submitted model had to be provided in TFLite format, with a file size smaller than 800 KB and an activation arena below 300 KB. This second limit refers to the memory required by the intermediate tensors during inference, and was particularly important because it directly affected the maximum complexity of the architecture. Moreover, the model could not

include operations unsupported by TensorFlow Lite for Microcontrollers, such as recurrent, custom, or Flex operations. For this reason, the design had to remain compact, memory-efficient, and compatible with the limited set of operations available on the target microcontroller.

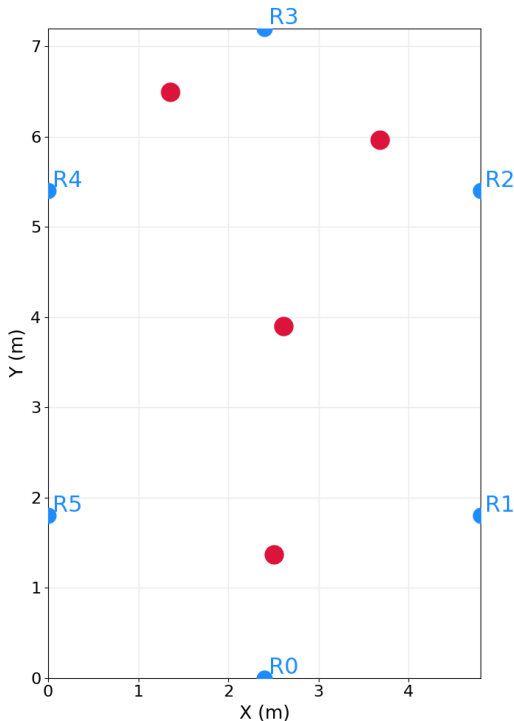


Fig. 1. View of the acquisition room, showing the coordinate system, the placement of the six UWB radars around the perimeter, and an example of people positions marked as red dots.

E. Evaluation score

The final evaluation score is computed as the F1 score. Predicted positions are first matched to ground-truth persons for each frame using the Hungarian algorithm. A matched prediction is considered a true positive only if its distance from the corresponding ground-truth position is less than or equal to 1 m. Predictions without a valid ground-truth match within this threshold are counted as false positives, while ground-truth persons without a valid prediction within the threshold are counted as false negatives. However, during our experiments, we also considered the average localization error and count accuracy over the test set as additional raw metrics to guide our design choices.

III. METHODS

This section describes the final pipeline used to train and export the model. It focuses on the final design choices, while the following section reports the main experiments performed during the project and explains how they influenced the final solution.

The final pipeline is composed of four main parts: data splitting, preprocessing, model architecture and training, and quantization/export.

A. Data Split

Each acquisition window contains 7500 frames and was divided into sub-windows of 50 consecutive frames. Since the radar signal is sampled at 25 Hz, each of them corresponds to 2 seconds of data. Each sub-window represents a sample for the model and the target associated with each sample is taken from the last frame of the corresponding sub-window, so that the model uses a short temporal context to predict the final people positions and presence mask.

The split was designed to reduce temporal leakage while preserving enough diversity in the three sets. A random split at frame or sub-window level would have produced highly correlated training and evaluation samples, because nearby frames belong to the same continuous acquisition. Conversely, assigning entire acquisition windows to train, validation, or test would have minimized leakage, but would also have made the splits less diverse, since only 24 complete windows are available and the occupancy levels are not equally represented.

For this reason, each acquisition window was divided into 10 contiguous macro-windows. Each macro-window contains 15 sub-windows, corresponding to 750 frames or 30 seconds of data. The train/validation/test split was then performed at macro-window level, using a 60% / 20% / 20% ratio and stratifying according to the number of people in the scene. This provided a compromise between avoiding leakage and keeping a representative distribution of scenarios across the three splits.

After the split, only the training set was balanced by over-sampling the under-represented occupancy classes. Validation and test sets were left unchanged, in order to evaluate the model on the original data distribution.

B. Preprocessing

The preprocessing pipeline starts from the raw `radar_cir_iq` array, where the CIR signal is represented through its In-phase and Quadrature components [4]. First, an exponential moving average background removal is applied to the raw I/Q data, using $\alpha = 0.96$. At each frame, the current background estimate is subtracted from the signal and then updated with the new observation. This step mainly reduces static and slowly-varying reflections from the environment, making the components related to people more evident.

For each 50-frame sub-window, the I/Q measurements are combined into the complex signal $\mathbb{I} + j\mathbb{Q}$, and a Fast Fourier Transform F is computed along the temporal dimension. For each radar and antenna, this produces a two-dimensional range-frequency map that describes both the range bins in which reflection occur and their variation over the 2-second interval [5].

Three input channels are then constructed from F : its real and imaginary components and its amplitude, computed as $A = |F| = \sqrt{(\Re(F))^2 + (\Im(F))^2}$. The real and imaginary channels retain signed, phase-related information, whereas the amplitude provides a direct, non-negative measure of the strength of each range-frequency component. Although the amplitude is mathematically derivable from the other two channels, including it explicitly avoids requiring the compact

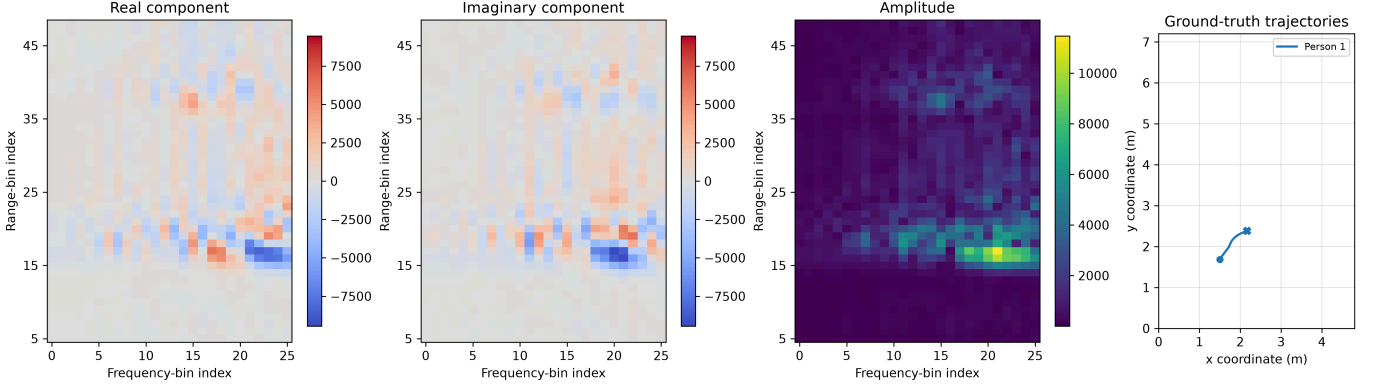


Fig. 2. Example of the three FFT-derived channels, namely real component, imaginary component, and amplitude, for radar 0 and antenna 0. The x-axis represents the frequency-bin index, the y-axis represents the selected range-bin index, and the color intensity represents the value of the corresponding FFT component. The ground-truth trajectory shows the movement of the person during the same 50-frame input window: this motion produces localized reflection patterns in the FFT maps, with stronger activations appearing in the respective range bins.

encoder to approximate this nonlinear operation. In our experiments, it improved presence and people-count estimation, motivating its processing through a dedicated, lighter encoder branch. The complementary use of complex I/Q and magnitude information for UWB device-free localization is also supported by previous work [1].

The three maps are stacked in the order real, imaginary, and amplitude for each radar and antenna. An example is shown in Fig. 2 for radar 0 and antenna 0: the x-axis represents the frequency-bin index, the y-axis represents the selected range-bin index, and the color intensity represents the value of the corresponding FFT component. To reduce the input size and focus on the most useful part of the signal, only range bins from 5 to 48 and frequency components from 0 to 25 are retained. This produces a compact two-dimensional representation of size 44×26 , which can then be processed with convolutional layers.

The final input sample has shape $(6, 3, 44, 26, 3)$, corresponding to six radars, three antennas per radar, 44 selected range bins, 26 selected frequency components, and three FFT-derived channels. Finally, the features are standardized using mean and standard deviation computed only on the training set. The same statistics are then applied to validation and test data, avoiding information leakage from the evaluation sets. A `STD_EPSILON` value of 10.0 is added to the standard deviation to avoid unstable scaling of low-variance features.

C. Model Architecture

The final model is based on a compact dual-encoder architecture designed to process the FFT representation described in the previous section. Each input sample has shape $(6, 3, 44, 26, 3)$, corresponding to six radars, three antennas per radar, a 44×26 bin-frequency map, and three channels: real part, imaginary part, and amplitude.

Before feature extraction, antenna dropout with rate 0.1 is applied during training. This operation is a regularization term that randomly removes antenna contributions and was introduced to make the model less dependent on a specific antenna and more robust to variations in the input signal.

The model first rearranges the input so that each radar is processed independently by the same convolutional encoders. In particular, the radar dimension is temporarily folded into the batch dimension, while the antenna information is concatenated into the channel dimension. Therefore, for each radar, the convolutional encoders receive a 2D bin-frequency map whose channels contain the information from the three antennas.

The input is then separated into two branches. The complex branch processes the real and imaginary FFT components of the three antennas, resulting in six input channels. The amplitude branch processes the amplitude component of the three antennas, resulting in three input channels. This separation allows the model to treat phase-related information and signal intensity information differently.

Both branches use three convolutional blocks composed of Conv2D, batch normalization, ReLU activation, and max pooling. All convolutions use a 3×3 kernel. The complex branch uses 32, 64, and 128 filters, while the amplitude branch uses 16, 32, and 64 filters. The complex branch is therefore given higher capacity, since real and imaginary components contain richer information about the structure of the received signal. This choice was obtained through empirical tuning, after several training runs with different architectural configurations. From a theoretical point of view, the selected filter progression is suitable for this task because it progressively extracts more abstract spatial-frequency patterns while keeping the model compact.

After the convolutional encoders, each radar representation is compressed into a compact feature vector. The pooling stage combines Top-K average pooling with $k = 6$ and soft attention pooling with temperature 4.0. Top-K pooling extracts the strongest activations from the feature map, while soft attention pooling learns a weighted combination of the most relevant spatial-frequency regions. The two pooled vectors are concatenated, producing one token per radar for each branch.

The complex and amplitude representations are first projected to the same dimension, equal to $D_ATTN \times N_HEADS = 32 \times 4 = 128$. The six projected complex vectors and six projected amplitude vectors are then concatenated

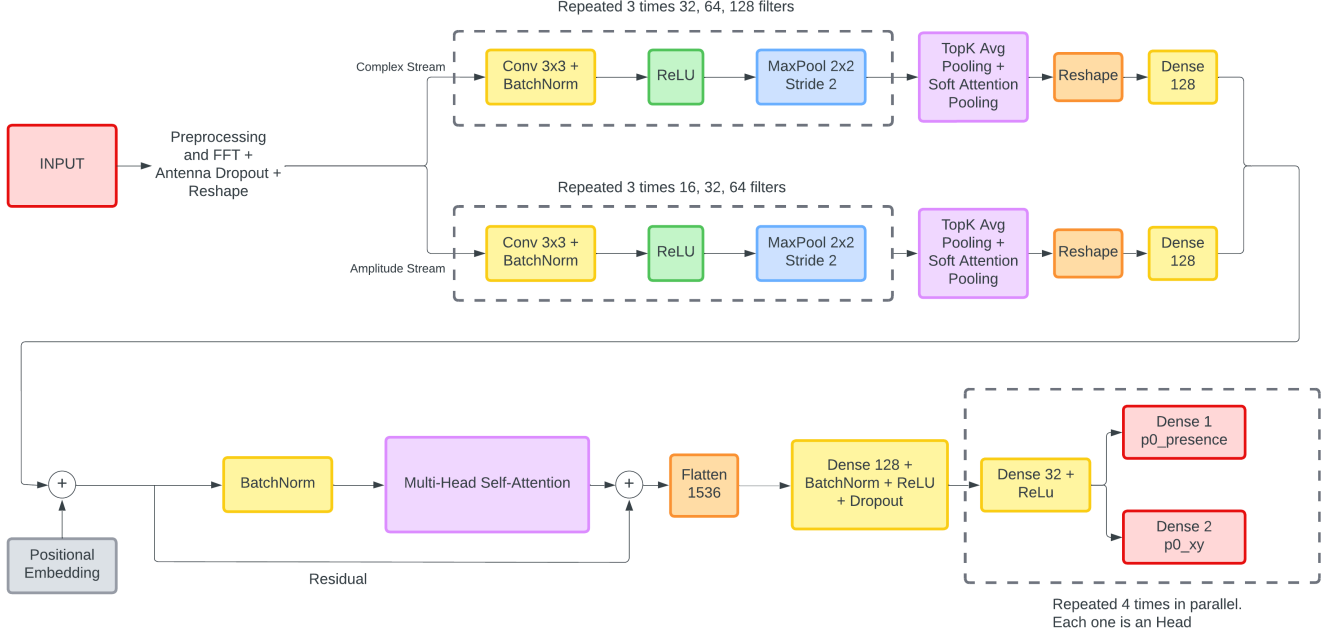


Fig. 3. Model architecture diagram. The encoder (top half) is shared across the radars, producing 2 tokens per radar which gets fused by an attention layer and fed to the detection heads (bottom half).

along the token dimension, producing a sequence of 12 tokens with shape $(12, 128)$. The tokens are ordered by branch and radar: the first six correspond to the complex representations of the six radars, while the remaining six correspond to their amplitude representations. A fixed position-specific embedding of shape $(12, 128)$ is added to distinguish the radar and branch associated with each token.

Then, after batch normalization, the 12 tokens are processed by one lightweight multi-head self-attention layer with 4 heads and a per-head attention dimension of 32. This layer allows the model to fuse information across the different radars and between the complex and amplitude representations. The outputs of the four heads are combined and projected back to a dimension of 128. A residual connection is used around the attention layer, helping to preserve the original token information.

The output of the attention block is flattened and passed through a shared dense block with 128 units, batch normalization, ReLU activation, and dropout equal to 0.3. This shared representation is then used by four independent prediction heads, one for each possible person slot. Each head contains a dense layer with 32 units and produces two outputs: a 2D position (x, y) and a presence logit. The position output estimates the location associated with that slot, while the presence logit indicates whether the slot corresponds to an actual person or should be ignored during post-processing. In this way, the model handles the variable number of people in the room through four fixed prediction slots, each of which can be activated or deactivated independently.

D. Loss Function and Output Matching

The model predicts four output slots, but the people in the room do not have a fixed order. Therefore, the loss cannot directly compare slot 0 with ground-truth person 0, slot 1 with ground-truth person 1, and so on. Before computing the loss, the predicted slots must be assigned to the ground-truth slots.

For this reason, the training objective is based on Hungarian matching. For each sample, a 4×4 cost matrix is built between predicted slots and ground-truth slots. The cost combines two terms: the squared localization error between predicted and ground-truth coordinates, and the binary cross-entropy cost associated with the presence prediction. The Hungarian algorithm is then used to find the assignment with minimum total cost.

After the assignment, the final loss is computed as:

$$L = L_{loc} + \lambda_{mask} L_{mask} \quad (1)$$

where $\lambda_{mask} = 1.5$.

The localization term L_{loc} is computed only on valid people, using a Charbonnier loss with $\epsilon = 0.1$. This term penalizes the distance between the matched predicted position and the corresponding ground-truth position, while being less sensitive to large errors than a purely quadratic loss [6].

The mask term L_{mask} is a binary cross-entropy loss applied to the four presence logits. It trains each output slot to predict whether it corresponds to an actual person or to an empty slot.

In summary, Hungarian matching is used to decide which prediction should be compared with each ground-truth person, while the optimized loss is composed of a Charbonnier localization term and a weighted binary cross-entropy presence

term. An additional count-classification loss is supported in the implementation, with the purpose of directly predicting the total number of people in the room to improve the count capabilities of the model. However, in the final configuration this term is disabled by setting its weight to zero, since it did not provide significant improvements in the experimental results. Therefore, the final model does not use a separate count-prediction head: the number of people is obtained from the four presence logits during post-processing.

E. Training Strategy

The final model is trained for 200 epochs with GPU acceleration. The first 5 epochs act as a warmup phase, linearly increasing the learning rate up to 0.001, then for the remaining epochs the learning rate decays with a cosine scheduler, this approach resulted in much stabler training than a constant learning rate. To help generalization, we trained with a weight decay of 0.0005 and a batch size of 16.

To improve the counting accuracy, we perform a tuning phase of the detection thresholds of each output head, maximizing the validation score. This proved particularly effective at minimizing the quantization degradation.

F. Optimizations

The final model was exported as a fully INT8 TensorFlow Lite model using post-training quantization. A representative subset of the training data was used during conversion to estimate the activation ranges required for integer quantization. Both model inputs and outputs were converted to INT8, making the exported model compatible with the deployment constraints.

Although quantization does not change the number of model parameters, it significantly reduces the memory footprint of the network by storing weights and activations with 8-bit integer precision instead of floating-point precision [7]. This is particularly important for edge deployment, where memory availability, bandwidth, and energy consumption are limited. Moreover, INT8 operations can be executed more efficiently on many embedded accelerators and microcontroller-oriented inference runtimes [7].

The architecture was also designed to limit the number of parameters before quantization. First, the same convolutional encoders are shared across all radars: the radar dimension is folded into the batch dimension, so the model does not learn six independent radar-specific encoders. This allows the network to process each radar separately while reusing the same filters, reducing redundancy and improving parameter efficiency.

The dual-branch structure also follows an efficient design. The complex branch is given higher capacity because it processes real and imaginary FFT components, which contain richer phase-related information, while the amplitude branch is intentionally lighter. This avoids assigning the same computational budget to all input components and keeps the model compact without discarding useful signal information.

Another important optimization is the use of pooling before the attention block. Top-K average pooling and soft attention pooling compress each per-radar feature map into

compact vectors before inter-radar fusion. As a result, the attention block operates only on 12 tokens, corresponding to six complex tokens and six amplitude tokens, instead of attending over the full spatial-frequency feature maps. This strongly limits the cost of the attention mechanism while still allowing the model to combine information across radars and feature branches.

Finally, the prediction stage is kept lightweight. The model uses a shared dense representation followed by four small prediction heads, one for each possible person slot. In the final configuration, no separate count-prediction head is used; the number of people is obtained directly from the four presence logits during post-processing. This avoids adding an additional output branch that did not provide significant experimental benefits, further reducing unnecessary model complexity.

Since quantization can change the behavior of the model, the final checkpoint was selected after conversion. The best floating-point checkpoints were first converted to INT8 and then evaluated again on the validation set. The checkpoint with the best post-quantization validation score was selected as the final model.

In the end, the submitted model has 449358 parameters and has an estimated activation arena of 266.6 KB. We further optimized the standardization parameters, as the original approach of computing the statistics for each combination of frequency, bin, antenna and radar in FP32 resulted in an excessive amount of SRAM required. Therefore we share the statistics among all of the different radars and save them in FP16, reducing the total amount of memory required by 12x. Additionally the tuned thresholds for the detection heads are saved as FP16, as in both cases INT8 quantization resulted in a steep performance drop, mandating for a variable quantization scheme. Standardization and threshold parameters are embedded in the model file as metadata, which results in a final model size of 551.2 KB, well under the 800 KB limit.

IV. EXPERIMENTS

The final model described above was obtained through many attempts and strategies, with the goal of finding an architecture able to accurately count and localize people while remaining lightweight with a limited number of parameters. This section summarizes the main experiments performed during the development of the model, highlighting the ideas behind them, the reasons why many of them did not work, and why some others were included in the final architecture.

A. Baseline Models

The first experiments were performed in the time domain. The initial baseline, implemented in `src/train_basic.py`, used a shared `Conv1D` encoder over the whole input window, after converting the raw I/Q signal into amplitude and phase. This model was useful to validate the training pipeline, the four-slot output format, and the presence prediction heads. However, it treated the radar, antenna, and range-bin dimensions mostly as a single flattened feature vector. This made the model simple, but it also removed part

of the structure of the problem before the network could exploit it.

For this reason, we then tested time-domain attention models. In the radar-level version, implemented in `src/radar_encoder/train_attn.py`, each radar was processed by the same convolutional encoder and the six radar embeddings were fused through an attention layer. The motivation was to preserve the identity of each radar and allow the model to combine different views of the room. We also tested an antenna-level version, implemented in `src/antenna_encoder/train_ant_attn.py`, where each antenna was encoded separately, the three antennas of each radar were fused first, and then the radar tokens were fused. This was theoretically useful because the antennas can contain angular information, but in practice the additional hierarchy increased complexity and did not give a better final trade-off than the radar-level approach.

We also tested a mean/variance antenna representation in `src/antenna_encoder/train_ant_mean_va.py`, compressing each 50-frame window into the temporal mean and variance of the amplitude. Variance was included because moving people produce localized increases in the variance of accumulated UWB CIR measurements, which can support device-free localization [8]. This reduced the input size, but it discarded too much temporal information. Since people are detected not only from reflection intensity but also from how the signal changes over time, this representation was not retained.

B. Frequency-domain Representations

After the first time-domain models, we moved to frequency-domain representations. This change was motivated by the fact that people affect the radar signal not only through the absolute reflected energy, but also through temporal variations inside the 50-frame window. A frequency-domain representation makes these variations more explicit and gives the convolutional encoder a structured range-frequency map instead of a raw temporal sequence [2].

The first FFT experiments converted the raw I/Q signal into amplitude and used the magnitude of its temporal FFT. They were implemented in `src/radar_encoder/train_attn_fft.py` and `src/antenna_encoder/train_ant_fft.py`. In these models, the FFT was computed along the temporal dimension of each 50-frame sub-window, and the resulting magnitude map was processed with Conv2D layers. These experiments showed that the frequency-domain formulation was more suitable than the original time-domain representation, because it separated static and dynamic signal components more clearly.

We then tested complex FFT inputs in `src/radar_encoder/train_cplx_fft.py` and `src/antenna_encoder/train_ant_cplx_fft.py`. Instead of keeping only the magnitude, these models used the real and imaginary parts of the FFT as channels. This preserved phase-related information, which can be useful for localization. We also tested adding the amplitude as a third channel. This improved the count behavior because it provided a direct measure of reflection strength, while the complex

channels preserved richer signal structure. For this reason, the final preprocessing in `src/train.py` keeps real part, imaginary part, and amplitude.

We also experimented with limiting the input range in both dimensions. These tests were supported by the `FFT_BIN_RANGE` and `FFT_FREQ_RANGE` options in `src/radar_encoder/train_attn_fft.py`, `src/radar_encoder/train_cplx_fft.py`, and the later dual-encoder scripts. The first range bins contain direct coupling and little useful localization information, while high range bins and unnecessary frequency components increase memory and computation. Cropping the input to bins 5..48 and frequencies 0..25 provided a compact representation while preserving the useful part of the signal, so this setting was fixed in `src/train.py`.

C. Matching and Background Removal

Once the slot-based output formulation was defined, we introduced the Hungarian matching in the loss. This was first tested in the baseline and attention scripts, including `src/train_basic.py`, `src/radar_encoder/train_attn.py`, and `src/antenna_encoder/train_ant_attn.py`, and was then used systematically in the FFT-based models. Since the four output slots have no fixed semantic order, a fixed slot-to-person assignment can penalize correct predictions only because they appear in a different order. Hungarian matching solved this issue by assigning predicted slots to ground-truth people according to the minimum combined localization and presence cost. This became part of the final training objective in `src/train.py`.

We also tested different ways of removing static reflections. A first approach, present in `src/train_basic.py`, `src/radar_encoder/train_attn.py`, and the early FFT scripts such as `src/radar_encoder/train_attn_fft.py`, subtracted a baseline computed from zero-person training windows. This was motivated by the idea that empty-room windows describe the static environment. However, this approach depends strongly on the availability and representativeness of zero-person data. We therefore tested exponential moving average background removal in `src/radar_encoder/train_cplx_fft.py` and later variants which proved to improve the stability of the learned features, so it was used in the final pipeline in `src/train.py`.

D. Architecture Variants

Also the choice of the final architecture derives from a series of attempts, evaluated around the FFT representation. A first complex FFT model, implemented in `src/radar_encoder/train_cplx_fft.py`, used a single convolutional encoder where real, imaginary, and amplitude channels were processed together. We then tested a dual-encoder design in `src/radar_encoder/train_cplx_fft_dual_encoder_fuse.py`, where the complex components and the amplitude were processed by separate convolutional encoders. This was motivated by the fact that phase-related information and sig-

nal intensity may require different filters. Two fusion strategies were compared. In `src/radar_encoder/train_cplx_fft_dual_encoder_fuse.py`, the complex and amplitude embeddings were concatenated for each radar before the attention layer. In `src/radar_encoder/train_cplx_fft_dual_encoder_tokens.py`, complex and amplitude embeddings were kept as separate tokens, producing 12 tokens in total. The second strategy performed better because it allowed the attention block to decide how to combine radar information and feature-type information jointly. This became the final architecture in `src/train.py`.

We also tested deeper attention stacks with multiple transformer blocks, mainly through the `TRANS_BLOCKS` option in `src/radar_encoder/train_attn_fft.py`, `src/radar_encoder/train_cplx_fft.py`, and `src/radar_encoder/train_cplx_fft_dual_encoder_tokens.py`. Although additional blocks increased the expressive power of the model, the number of tokens was already small and the dataset size was limited. Extra blocks increased parameters without a sufficient gain leading to overfitting, so the final model in `src/train.py` uses a single lightweight attention block.

Different convolutional blocks were also explored in `src/radar_encoder/train_cplx_fft_dual_encoder_tokens.py`, including standard `Conv2D`, depthwise-separable convolutions, separable convolutions, and MobileNetV2-style blocks. These layers are attractive because they reduce the number of parameters, but in our setting they did not improve the embedded trade-off, and some configurations increased the activation memory enough to become problematic. Standard `Conv2D` blocks were therefore kept in `src/train.py`.

E. Improving training dynamics

A lot of effort was spent in improving the training dynamics, to achieve better generalization and less quantization degradation without increasing the final model size. One of the first things that we tried, found in every FFT variant, was using a learning rate cosine scheduler as described in Section III.E, which proved to be successful. Furthermore we experimented with two augmentation techniques: the first randomly drops out an antenna to reduce the over-reliability on individual antennas, the second applies a constant random phase shift to all input components as it doesn't affect the amplitude, and the phase information lies in the phase differences among different bins, not their absolute values. The first proved to be effective with a small probability of 0.1, while the latter proved to be ineffective, so it was discarded for the final model.

From our experiments, the models had the most troubles at counting the correct number of people, especially in the 3 people cases. To address this, we tried various solutions:

- Giving more weight to the counting loss via the `MASK_LOSS_WEIGHT` option, which generally improved the performance

- Adding an additional train-only counting head, however this did not improve counting accuracy and led to worse localization
- Tuning the threshold of the detection heads on the validation set instead of using a constant 0.5, this proved to be very effective and is done by default by the final train script
- Pre-training the encoder or the full backbone at counting and then proceed with the standard training, this did not work out

We also experimented with different types of localisation losses to further improve performance. From the standard MSE loss, in `src/radar_encoder/train_cplx_fft_dual_encoder_tokens.py` we experimented with the Huber and Charbonnier losses. Both of them penalize large errors less than MSE and can potentially be more robust when the data contains outliers. From our experiments, Huber loss didn't provide any meaningful improvements, while the Charbonnier loss consistently improved localization without affecting counting performance.

To tackle the quantization sensitivity, we tried to add at the end of training a few epochs of QAT fine-tuning instead of the standard PTQ, via the QAT options in `src/radar_encoder/train_cplx_fft_dual_encoder_tokens.py`, this never reduced the degradation, and sometimes actually made the performance collapse during those epochs. What ended up working was to, instead of selecting the best validation checkpoint to quantize, select the top 5 via the `TOPK_PTQ_K` option and then select the best performing quantized checkpoint on the validation set.

Moreover, at the beginning, the FFT models used a global average pooling layer, for example in `src/radar_encoder/train_attn_fft.py`. However, we discovered that this choice was problematic because GAP compresses the whole range-frequency map into a single average vector, removing most of the spatial-frequency structure learned by the convolutional encoder. For this task, however, the position and strength of localized activations are important, since people are associated with specific reflection patterns rather than with a uniform response over the whole map. We therefore replaced GAP with Top-K pooling and soft attention pooling in `src/radar_encoder/train_cplx_fft.py` and `src/radar_encoder/train_cplx_fft_dual_encoder_tokens.py`, keeping this choice in the final model.

V. RESULTS

We evaluated the model performance on our test set, as the example provided in the assignment is part of the train and validation sets, making it not suitable for evaluation. We compared the performance before and after quantization, comparing also the resources required to run the model. The results of our analysis can be found in Table II.

TABLE II
PERFORMANCE METRICS BEFORE AND AFTER QUANTIZATION.

	FP32 Model	Int8 Model
Localization Error (m)	0.4766	0.4862
Count Accuracy (%)	87.1	86.7
F1 score	0.8281	0.8242
Model file size (KB)	1976.31	551.2
Activation arena (KB)	1023	266

The final quantized model presents minimal performance degradation, with the final F1 score dropping by only 0.47%, while reducing the model file size by 3.59 times. Note that in the FP32 model standardization and threshold parameters are not included in the model file, while in the quantized model they are and occupy approximately 6.9% of the size. Quantization additionally reduces the activation arena size of 3.84 times, making it possible to run the model with the given constraints.

VI. DISCUSSION

UWB radars produce noisy data by nature, mandating heavy preprocessing otherwise models cannot extract meaningful information. Still, even with our preprocessing pipeline sometimes models have a hard time properly understanding the scene, due to the limited size and variety of the dataset. The size problem became apparent early and persisted, as scaling up the model never seemed to work. The variety problem instead impacted how the train, validation and test split was made, leading us to the macro-window solution presented in Section III.A. This meant that a hypothetical distillation pipeline where we first train a big model, then distill it to a smaller model was less promising, which made us focus on preprocessing, architectural and training techniques to extract as much signal from the noisy data.

The biggest performance improvements came from using first the FFT as input, and then the complex components of the FFT. The latter especially helped with the localization error, however doing so at first came at a cost in counting accuracy. We suspect that the model could easily extract the number of people from the amplitude of the FFT, but had troubles doing so from the noisier complex components. We tried to make the encoder wider and deeper, as theoretically it could learn to compute the amplitude from the complex components, but it ended up failing due to the aforementioned dataset limitations. This lead us to investigate on how to give the amplitude to the model, leading us to the dual token approach, which even if it increases the size and amount of computation, we think is worth the performance gains.

In the end, we were able to produce a model with strong performances while respecting the given constraints with minimal quantization degradation, thanks to a lot of effort spent in stabilizing the training to produce robust models. A breakdown of the F1 score based on the count of people in Table III. Note that due to the evaluation setup with an empty

room the F1 cannot be higher than 0.0, however our model correctly identifies that case 88.5% of the times.

TABLE III
TEST F1 BASED ON THE NUMBER OF PEOPLE.

Number of people	Average F1
0	0.00
1	0.98
2	0.94
3	0.80
4	0.85

The main limitations come from the 3-people scenarios, where the model tends to hallucinate an additional fake person, however the performance in that case is still acceptable with a score of 0.80.

VII. CONCLUSIONS

In summary, the final solution combines EMA background removal, a complex temporal FFT representation, two separate encoders for complex channels and amplitude and an attention-based mechanism across the six radar. The resulting INT8 model achieves an average localization error of 0.4862m and a count accuracy of 86.7%. This turns in the final evaluation in an F1 score of 0.8242, while satisfying both the model-size and activation-memory constraints. Overall, the experiments showed that carefully preserving and processing the structure of the radar signals was more effective than simply increasing model complexity.

The main limitation remains the size and variety of the available dataset, so future work could investigate richer data augmentation such that scaling up the model is effective, with the possibility of a distillation pipeline for a more robust model.

Another direction concerns quantization. The tested QAT strategy was unstable and did not improve the final results. Future work could investigate more advanced solutions. These techniques could further reduce the quantization error and the performance degradation introduced by INT8 conversion, while preserving the memory and computational advantages required for embedded deployment.

The preprocessing pipeline could also be improved through more adaptive background-removal methods and a more detailed analysis of the useful range and frequency components. Finally, the amplitude information could be integrated more efficiently. The current dual-encoder solution performs well without exceeding the provided constraints; however, a cleaner integration of the amplitude channel could reduce architectural complexity and computational cost while preserving its contribution to counting and localization performance.

REFERENCES

- [1] M. Cimdins, S. O. Schmidt, and H. Hellbrück, "Comparison of I/Q- and Magnitude-Based UWB Channel Impulse Responses for Device-Free Localization," in *2021 International Conference on Localization*

- and GNSS (ICL-GNSS), IEEE, 2021, pp. 1–7. doi: 10.1109/ICL-GNSS51451.2021.9452299.
- [2] M. Pavan, A. Caltabiano, and M. Roveri, “TinyML for UWB-radar based presence detection,” in *2022 International Joint Conference on Neural Networks (IJCNN)*, 2022, pp. 1–8. doi: 10.1109/IJCNN55064.2022.9892925.
 - [3] G. Lee, S. An, B.-J. Jang, and S. Lee, “Deep Learning for Counting People from UWB Channel Impulse Response Signals,” *Sensors*, vol. 23, no. 16, p. 7093, 2023, doi: 10.3390/s23167093.
 - [4] H. H. Y. Shalby, “exercise session 5, Embedded and Edge AI course.” 2026.
 - [5] I. Brishtel, S. Krauss, M. Chamseddine, J. R. Rambach, and D. Stricker, “Driving Activity Recognition Using UWB Radar and Deep Neural Networks,” *Sensors*, vol. 23, no. 2, p. 818, 2023, doi: 10.3390/s23020818.
 - [6] J. T. Barron, “A General and Adaptive Robust Loss Function,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 4331–4339. doi: 10.1109/CVPR.2019.00446.
 - [7] R. Krishnamoorthi, “Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper,” *arXiv preprint arXiv:1806.08342*, 2018, [Online]. Available: <https://arxiv.org/abs/1806.08342>
 - [8] A. Ledergerber and R. D’Andrea, “A Multi-Static Radar Network with Ultra-Wideband Radio-Equipped Devices,” *Sensors*, vol. 20, no. 6, p. 1599, 2020, doi: 10.3390/s20061599.